

```

#include "raylib.h"
#include < stdlib . h >
#define LARGURA 800
#define ALTURA 600
#define QUANTIDADE 20
#define PONTUACAO_MAX 100
typedef struct {
char nome [ 16 ] ;
int pontuacao ;
} Placar ;
Placar * criarPlacares ( int quantidade ) {
Placar * placares = ( Placar * ) malloc ( quantidade * sizeof ( Placar ) ) ;
for ( int i = 0 ; i < quantidade ; i ++ ) {
Placar * p = ( placares + i ) ;
TextCopy ( p -> nome , TextFormat ( "J%02d" , i + 1 ) ) ;
p -> pontuacao = GetRandomValue ( 10 , PONTUACAO_MAX ) ;
}
return placares ;
}
void ordenarBubbleSort ( Placar * vetor , int n , long * comparacoes , long * trocas ) {
* comparacoes = 0 ;
* trocas = 0 ;
for ( int i = 0 ; i < n - 1 ; i ++ ) {
for ( int j = 0 ; j < n - 1 - i ; j ++ ) {
( * comparacoes ) ++ ;
if ( vetor [ j ] . pontuacao > vetor [ j + 1 ] . pontuacao ) {
Placar temp = vetor [ j ] ;
vetor [ j ] = vetor [ j + 1 ] ;
vetor [ j + 1 ] = temp ;
( * trocas ) ++ ;
}
}
}
}
void ordenarInsertionSort ( Placar * vetor , int n , long * comparacoes , long * trocas ) {
* comparacoes = 0 ;
* trocas = 0 ;
for ( int i = 1 ; i < n ; i ++ ) {
Placar chave = vetor [ i ] ;
int j = i - 1 ;
while ( j >= 0 ) {
( * comparacoes ) ++ ;
if ( vetor [ j ] . pontuacao > chave . pontuacao ) {
vetor [ j + 1 ] = vetor [ j ] ;
( * trocas ) ++ ;
j -- ;
} else {
break ;
}
}
vetor [ j + 1 ] = chave ;
}
}
int buscaSequencial ( Placar * vetor , int n , int alvo , long * comparacoes ) {
* comparacoes = 0 ;
for ( int i = 0 ; i < n ; i ++ ) {
( * comparacoes ) ++ ;
if ( vetor [ i ] . pontuacao == alvo ) {
return i ;
}
}
return - 1 ;
}
int buscaBinaria ( Placar * vetor , int n , int alvo , long * comparacoes ) {
* comparacoes = 0 ;
int inicio = 0 ;
int fim = n - 1 ;
while ( inicio <= fim ) {
( * comparacoes ) ++ ;
int meio = ( inicio + fim ) / 2 ;
if ( vetor [ meio ] . pontuacao == alvo ) {
return meio ;
}
if ( vetor [ meio ] . pontuacao < alvo ) {
inicio = meio + 1 ;
} else {
fim = meio - 1 ;
}
}
return - 1 ;
}
int main ( void ) {
InitWindow ( LARGURA , ALTURA , "Atividade 7 - Busca e Ordenacao" ) ;
SetTargetFPS ( 60 ) ;
Placar * placares = criarPlacares ( QUANTIDADE ) ;
long comparacoes = 0 ;
long trocas = 0 ;
double tempoMs = 0.0 ;
int ordenado = 0 ;
int alvo = 50 ;
int resultado = - 1 ;
while ( ! WindowShouldClose ( ) ) {
if ( IsKeyPressed ( KEY_B ) ) {
double inicio = GetTime ( ) ;

```

```

ordenarBubbleSort (
placares ,
QUANTIDADE ,
& comparacoes ,
& trocas
) ;
double fim = GetTime ( ) ;
tempoMs = ( fim - inicio ) * 1000.0 ;
ordenado = 1 ;
}
if ( IsKeyPressed ( KEY_I ) ) {
ordenarInsertionSort (
placares ,
QUANTIDADE ,
& comparacoes ,
& trocas
) ;
ordenado = 1 ;
}
if ( IsKeyPressed ( KEY_S ) ) {
resultado = buscaSequencial (
placares ,
QUANTIDADE ,
alvo ,
& comparacoes
) ;
}
if ( IsKeyPressed ( KEY_W ) && ordenado ) {
resultado = buscaBinaria (
placares ,
QUANTIDADE ,
alvo ,
& comparacoes
) ;
}
BeginDrawing ( ) ;
ClearBackground ( RAYWHITE ) ;
DrawText ( "B: Bubble Sort" , 20 , 20 , 20 , BLACK ) ;
DrawText ( "I: Insertion Sort" , 20 , 45 , 20 , BLACK ) ;
DrawText ( "S: Busca Sequencial" , 20 , 70 , 20 , BLACK ) ;
DrawText ( "W: Busca Binaria" , 20 , 95 , 20 , BLACK ) ;
DrawText (
TextFormat ( "Comparacoes: %ld" , comparacoes ) ,
20 , 130 , 20 , BLACK
) ;
DrawText (
TextFormat ( "Trocas: %ld" , trocas ) ,
20 , 155 , 20 , BLACK
) ;
DrawText (
TextFormat ( "Tempo Bubble Sort: %.4f ms" , tempoMs ) ,
20 , 180 , 20 , BLACK
) ;
DrawText (
TextFormat ( "Alvo: %d" , alvo ) ,
20 , 205 , 20 , BLACK
) ;

DrawText (
TextFormat ( "Resultado: %d" , resultado ) ,
20 , 230 , 20 , BLACK
) ;
for ( int i = 0 ; i < QUANTIDADE ; i ++ ) {
int altura = placares [ i ] . pontuacao * 3 ;
int x = 20 + i * 38 ;
int y = ALTURA - altura - 30 ;
DrawRectangle ( x , y , 30 , altura , BLUE ) ;
DrawText (
TextFormat ( "%d" , placares [ i ] . pontuacao ) ,
x , y - 20 , 12 , BLACK
) ;
}
EndDrawing ( ) ;
}
free ( placares ) ;
CloseWindow ( ) ;
return 0 ;
}

```